**Ramificaciones de GitHup**

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

# CAPÍTULO 3. RAMIFICACIONES EN GIT

## Introducción

Las ramificaciones en Git, también conocidas como branches, son una herramienta que permite desarrollar diferentes versiones o funcionalidades de un proyecto de manera independiente, sin modificar directamente la rama principal. Esta característica facilita el trabajo colaborativo, la corrección de errores, la implementación de nuevas funciones y la experimentación con diferentes soluciones. Además, permite integrar los cambios realizados en distintas ramas mediante procesos de fusión. En este capítulo se explican los conceptos fundamentales de las ramificaciones, la creación y administración de ramas, los procedimientos de fusión, el trabajo con ramas remotas y la reorganización del historial mediante el comando git rebase.

### 3.1. ¿Qué es una rama?

Una rama  es una línea independiente de desarrollo dentro de un repositorio Git. Permite realizar modificaciones sin afectar directamente el trabajo que se encuentra en otras ramas. Por ejemplo, en un proyecto de desarrollo de software se puede utilizar una rama principal para conservar una versión estable y crear otras ramas para desarrollar nuevas funciones o solucionar errores. En Git, la rama principal se ha llamado tradicionalmente master, aunque actualmente es frecuente utilizar el nombre main.

**Características principales de las ramas:**

- Permiten trabajar en diferentes funcionalidades de manera independiente
- Facilitan la corrección de errores sin modificar la versión principal
- Ayudan a organizar el trabajo de los integrantes de un equipo
- Permiten integrar cambios cuando el trabajo está terminado

- Facilitan la experimentación sin afectar otras versiones del proyecto.

Una rama funciona como un indicador que señala una confirmación específica del historial.

### 3.2. Procedimientos básicos para ramificar y fusionar

La ramificación y la fusión son dos procesos fundamentales para trabajar con diferentes versiones de un proyecto. Crear una rama nueva:

- Para crear una rama se utiliza el comando: **git branch nombre_de_la_rama**

  Por ejemplo: **git branch desarrollo**

  Este comando crea una rama llamada desarrollo, pero no cambia automáticamente a ella

- Cambiar de rama:
- Para cambiar a una rama existente se utiliza: **git checkout nombre_de_la_rama**

  Por ejemplo: **git checkout** desarrollo o **git switch desarrollo**
- Crear y cambiar a una rama en un solo paso: **git checkout -b desarrollo** o **git switch -c desarrollo**

  Estos comandos crean la nueva rama y permiten comenzar a trabajar directamente en ella

- Fusionar ramas:
- Cuando el trabajo de una rama está terminado y se desea integrar en otra, se utiliza el comando: **git merge nombre_de_la_rama**
- Para integrar los cambios de desarrollo en main, primero se debe cambiar a la rama principal: **git switch main**
- Después se ejecuta: **git merge desarrollo**

De esta manera, Git incorpora los cambios de la rama desarrollo a la rama main.

- Eliminar una rama:
- Se puede eliminar utilizando: **git branch -d desarrollo**

  Este comando permite eliminar una rama que ya ha sido fusionada de manera segura

### 3.3. Gestión de ramas

La gestión de ramas consiste en organizar, consultar, crear, cambiar y eliminar las diferentes líneas de desarrollo de un proyecto

- Consultar las ramas existentes: **git branch**

  Muestra las ramas locales del repositorio y señala cuál se encuentra activa.
- Crear una rama: **git branch nombre_de_la_rama**

  Permite crear una nueva rama sin cambiar a ella
- Cambiar de rama: **git switch nombre_de_la_rama**

  Permite seleccionar la rama en la que se desea trabajar.
- Cambiar el nombre de una rama: **git branch -m nuevo_nombre**

  Este comando modifica el nombre de la rama actual.
- Eliminar una rama: **git branch -d nombre_de_la_rama** como lo vimos mas atrás

  Elimina una rama que ya no se necesita y que ha sido integrada
- Consultar las ramas junto con sus últimas confirmaciones: **git branch -v**

  Muestra información resumida de las ramas y sus confirmaciones más recientes.

La correcta gestión de las ramas permite mantener el proyecto organizado y facilita el seguimiento de las modificaciones realizadas por los colaboradores

### 3.4. Flujos de trabajo ramificados

Los flujos de trabajo ramificados son formas organizadas de utilizar las ramas para desarrollar proyectos de manera individual o colaborativa. Su objetivo es distribuir las tareas, mantener el código organizado y evitar que los cambios de diferentes funcionalidades interfieran entre sí.

- Algunos flujos de trabajo comunes son:

1. **Ramas de largo recorrido:**

Consisten en mantener varias ramas activas durante un periodo prolongado. Por ejemplo, una rama principal puede contener la versión estable del proyecto, mientras que otra rama se utiliza para desarrollar nuevas funciones y realizar pruebas

2. **Ramas por funcionalidad:**

Se crea una rama independiente para desarrollar cada nueva característica del proyecto. Cuando la funcionalidad está terminada y revisada, sus cambios se integran en la rama principal

3. **Ramas para corregir errores:**

Se utilizan para solucionar problemas específicos sin interrumpir el desarrollo de otras funcionalidades

4. **Ventajas de los flujos de trabajo ramificados:**

- Permiten distribuir las actividades entre diferentes colaboradores
- Facilitan la revisión de los cambios antes de integrarlos
- Evitan modificar directamente la versión estable del proyecto
- Mejoran la organización y el seguimiento de las tareas
- Facilitan la identificación y corrección de errores

### 3.5. Ramas remotas

Las ramas remotas son referencias a las ramas que existen en repositorios alojados en servidores o plataformas como GitHub. Permiten compartir el trabajo realizado y mantener sincronizados los cambios entre diferentes colaboradores

- Consultar las ramas remotas: **git branch -r**

  Muestra las ramas remotas conocidas por el repositorio local

- Consultar todas las ramas: git branch -a

  Permite visualizar tanto las ramas locales como las remotas

- Enviar una rama al repositorio remoto: **git push origin nombre_de_la_rama**

  Este comando envía la rama local al repositorio remoto identificado como origin

- Obtener información actualizada del repositorio remoto: **git fetch**

  Descarga información y referencias actualizadas del repositorio remoto, pero no integra automáticamente esos cambios en la rama local actual

- Crear una rama local que siga una rama remota: **git switch --track origin/nombre_de_la_rama**

  Permite crear una rama local vinculada a una rama remota existente

- Eliminar una rama remota: **git push origin --delete nombre_de_la_rama**

  Este comando elimina la rama indicada del repositorio remoto

El uso de ramas remotas es especialmente importante en proyectos colaborativos, ya que permite compartir avances y trabajar de manera organizada con otros integrantes

### 3.6. Reorganizar el trabajo realizado

La reorganización, conocida como *rebase*, es un procedimiento que permite aplicar nuevamente las confirmaciones de una rama sobre otra base diferente. Git ofrece dos formas principales de integrar los cambios de distintas ramas: la fusión (*merge*) y la reorganización (*rebase*),

La fusión combina los cambios de dos ramas y puede generar una confirmación especial de fusión. En cambio, la reorganización vuelve a aplicar las confirmaciones de una rama sobre otra, generando un historial diferente y generalmente más lineal. Comando para realizar una reorganización: **git rebase nombre_de_la_rama**

- Por ejemplo:
- **git switch desarrollo**
- **git rebase main**

Este procedimiento aplica los cambios de la rama desarrollo sobre la versión actual de main

- **Ventajas de utilizar rebase:**
- Permite organizar el historial de confirmaciones
- Facilita la lectura de la secuencia de cambios
- Ayuda a integrar modificaciones de diferentes ramas
- Puede mantener un historial de desarrollo más lineal

- **Precaución al utilizar rebase:**

No se recomienda reorganizar ramas que ya han sido compartidas y utilizadas por otros colaboradores sin coordinar previamente los cambios, porque este procedimiento

modifica los identificadores de las confirmaciones. Por esta razón, es importante comprender el funcionamiento de git rebase antes de utilizarlo en proyectos colaborativos

### 3.7. Ejercicio práctico: creación y fusión de ramas

El siguiente ejercicio permite aplicar los conocimientos adquiridos mediante un repositorio de Git

**Procedimiento:**

1. Abrir Git Bash o CMD
2. Ingresar a la carpeta del proyecto
3. Consultar las ramas existentes: **git branch**
4. Crear una nueva rama llamada desarrollo: **git branch desarrollo**
5. Cambiar a la nueva rama: **git switch desarrollo**
6. Crear un archivo llamado **prueba.txt** dentro del proyecto
7. Agregar el archivo al área de preparación: **git add prueba.txt**
8. Registrar los cambios: **git commit -m "Crea archivo de prueba en rama desarrollo"**
9. Regresar a la rama principal: **git switch main**
10. Fusionar los cambios de la rama desarrollo: **git merge desarrollo**
11. Consultar las ramas disponibles: **git branch**
12. Si ya no se necesita la rama de desarrollo, eliminarla: **git branch -d desarrollo**

Con esto buscamos comprender cómo crear ramas, cambiar entre ellas, registrar modificaciones y fusionar los cambios en la rama principal de un proyecto

## Conclusiones

Las ramificaciones constituyen una de las funcionalidades más importantes de Git, porque permiten trabajar en diferentes partes de un proyecto sin afectar directamente su versión principal. Los comandos **git branch, git switch, git merge y git rebase** facilitan la organización de las actividades, la integración de modificaciones y el control del historial de desarrollo. Además, las ramas remotas permiten compartir el trabajo mediante plataformas como GitHub, favoreciendo la colaboración entre los integrantes de un equipo.

## Referencias bibliográficas


1. Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2
2. Git. (s. f.). *Ramificaciones en Git*. https://git-scm.com/book/es/v2/Ramificaciones-en-Git
3. Git. (s. f.). *Procedimientos básicos para ramificar y fusionar*. https://git-scm.com/book/